Express.js

Node.js Express

This module assumes that students understand the following:

1. ASE 220 Topics: MongoDB

- We discuss MongoDB and Mongoose in `mongodb_and_mongoose`

2. ASE 230 Topics: REST API & Server Side Application Development

- PHP/Laravel and JavaScript/AJAX

What Node.js Is

- JavaScript runtime (V8 + libuv)
- Your **server runs inside Node**, not Apache/Nginx
- Persistent process instead of “run PHP per request”

PHP model:

Request → PHP runs → exits

Node model:

Server always running → handles many requests

What Express Is (Compared to PHP/Laravel)

- Minimal web framework for Node.js
- Similar to **Laravel routing layer**
- Provides:
  - Routing
  - Middleware
  - Request/response helpers
- Does **not** include:
  - ORM
  - Templating system (optional)
  - Auth scaffolding

Routing Comparison

PHP (Laravel)

```php
Route::get('/posts', function () {
    return Post::all();
});
```

Node.js (Express)

```javascript
app.get('/posts', async (req, res) => {
  const posts = await postsCol.find().toArray();
  res.json(posts);
});
```

Same REST concept, different environment.

Request Model

PHP

- Web server invokes PHP per request
- Clean fresh state on each request

Node.js

- Express server stays running
- Can keep:
  - DB connections
  - Caches
  - Variables in memory

Great for:

- WebSockets (We will use WebSocket in the project)
- Streaming
- Live apps

Middleware

Middleware = functions that run before your route handler.

Equivalent to Laravel middleware, but simple to use without `php artisan make:middleware`.

```
app.use(express.json());
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});
```

Laravel → Middleware + Service Providers + Filters

Express → Everything is middleware

Views (EJS)

Using EJS (like Blade):

```
app.get('/', (req, res) => {
  res.render('home', { posts: posts });
});
```

This is the EJS template (`home.ejs`).

```
<ul class="list-group">
<% for (var i = 0; i < posts.length; i ++) { %>
    <h4><%= posts[i].title %></h4>
    <p><%= posts[i].date %></p>
<% } %>
</ul>
```

Folder Structure (Laravel-Like)

```
/routes
/controllers
/models
/views
```

Express does NOT enforce structure, but this is common.

Express Example (index_express.js)

We use MongoDB as the backend Database.

```
const { getDB } = require('./util/db');

const DATABASE = 'todoapp';
let db;

async function initialize() {
  db = await getDB(DATABASE);
}
initialize();
```

- In CommonJS, we cannot use `await` at the top level, so we put the `await` code in an `async` function (`initialize`).
- Refer to `mongodb_and_mongoose` for using MongoDB.

Express application

```
const express = require('express');
const app = express();
```

- `express` is the Express framework function.
- Calling `express()` creates an application object.

`app` is the Express application that processes REST API requests and sends responses.

Using Middleware

This middleware allows Express to parse data from HTML forms (sent using POST, PUT, PATCH, or DELETE).

```
app.use(express.urlencoded({ extended: true }))
```

It parses requests with `Content-Type: application/x-www-form-urlencoded`.

This is the default format used by:

- `<form method="POST">` in HTML
- Standard browser form submissions
- Simple API clients

After using it, Express places the parsed values into: `req.body`

Body Parsing

Without these two middleware functions, Express cannot read `req.body` .

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

- `express.json()` : parses JSON request bodies
- `express.urlencoded()` : parses URL-encoded form data (e.g., HTML `<form>` submissions)
- Both put the parsed data into `req.body`

Callback functions

In this example, the server processes GET request to '/'.

```
app.get('/', function (req, resp) {
  resp.sendFile(__dirname + '/write.html')
});
```

- When the client goes to http://localhost:5500/
- Express runs this handler
- It sends the file write.html back to the browser
- This is usually a form page that submits data

The server processes POST request of add API.

```
app.post('/add', async function (req, resp) {
  try {
    const posts = db.collection(COLLECTION);
    const query = { title: req.body.title, date: req.body.date };
    await posts.insertOne(query);
    resp.send('Stored to MongoDB OK');
  } catch (e) {
    console.error(e);
    resp.status(500).send('Error: could not store the data');
  }
});
```

1. Waits for a POST submission
2. Reads the form data from `req.body`
3. Inserts the data into MongoDB
4. Sends a message back to the client (or status 500 if the insert failed)

The Request method and endpoint

The method of the Express `app` object matches the request method, and the first argument matches the endpoint (API path).

```
app.get('/', function); // GET method / root
app.post('/add', function); // POST method /add API
app.delete('/delete', function); // DELETE method /delete API
```

runAddPost function

We separate the routing from the business logic (the algorithm) to make the code easier to read and maintain.

```
app.post('/add', function(req, resp) {
    runAddPost(req, resp);
});

async function runAddPost(req, resp) {
  try {
    ...
  }
```

In this example, we use `counter` collection to keep track of the ID.

The JSON representation of the document is as follows:

```
{
  "name": "count",
  "count": N
}
```

There is only one document in this collection. We find it with `name = "count"`, and the current count (N) is the value of the `count` field.

```
async function runAddPost(req, resp) {
  try {
    const counter = db.collection(COUNTER);
    const result = await counter.findOneAndUpdate(
      { name: 'count' },
      { $inc: { count: 1 } },
      { returnDocument: 'after', upsert: true }
    );
    ...
  }
```

1. `$inc: { count: 1 }`: Increments the `count` field by 1 (creates it if missing).
2. `returnDocument: 'after'`: Returns the document after the update.
3. `upsert: true`: Inserts a new document if none matches the filter.

Using the new ID (`newId`), we create a post from the title and date in `req.body` and insert it into the `posts` collection.

```
    const newId = result.count; // now safe to access
    const newPost = {
      _id: newId,
      title: req.body.title,
      date: req.body.date
    };
    await posts.insertOne(newPost);

    resp.send('Stored to MongoDB OK');

  } catch (e) {
    console.error(e);
    resp.status(500).send('Error: could not store the data');
  }
```

We return the string 'Stored to MongoDB OK' to the client (or status 500 if something goes wrong).

Start the Server

```
app.listen(5500, function () {
  console.log('listening on 5500')
});
```

- The client can access the server from http://localhost:5500/.
- We can change the port number by replacing 5500 with any port number.